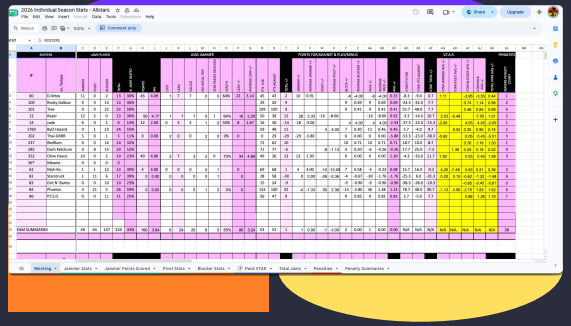
DEVFEST DC 2026

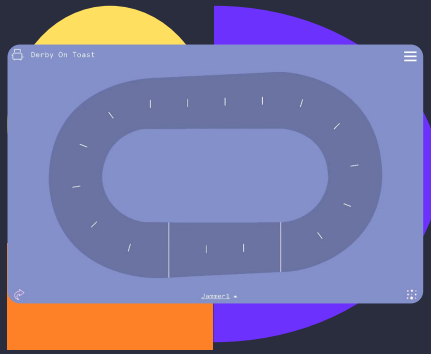
Two Tablets, One Bench

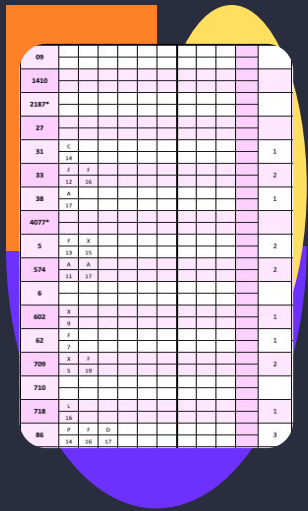
Real-Time Multiplayer at the Edge with Durable Objects and Workers AI

Amanda J Kendal-Brown
Senior Front-End Engineer · Capital One

Coaches can see it. They can't track it.







Post Game Stats

Officials provide these after the game. Teams tracking their own use Excel or Sheets. Either way — not live.

Derby on Toast

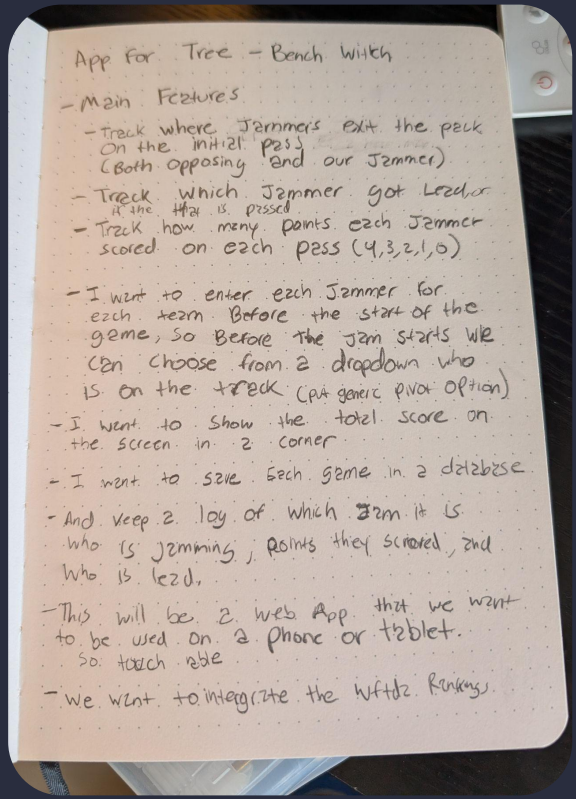
A web app built on heatmap.js. Single device, no persistence. Now defunct

Live Penalties

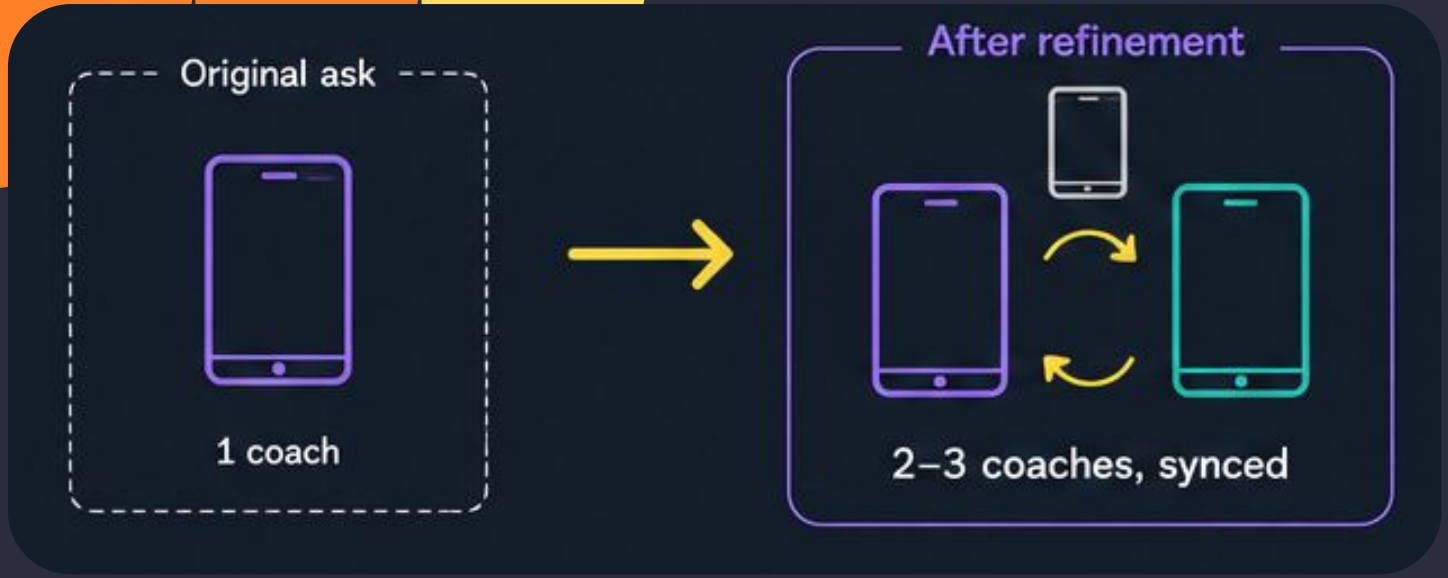
Some tournaments only. Requires Derby Scoreboard, a stats crew, and a running E-P-L-T. Not universal

Playoffs started in 30 days.

My Notebook.


App for Tree - Bench Witch
- Main Features
- Track where Jammers exit the pack on the initial pass (Both opposing and our Jammer)
- Track which Jammer got Lead or if the Hat is passed
- Track how many points each Jammer scored on each pass (4,3,2,1,0)
- I want to enter each Jammer for each team Before the start of the game, so Before the jam starts we can choose from a dropdown who is on the track (put generic pivot option)
- I want to show the total score on the screen in a corner
- I want to save each game in a database
- And keep a log of which Jam it is who is jamming, points they scored, and who is lead,
- This will be a web App that we want to be used on a phone or tablet. So touch able
- we want to integrate the WFTDA Rankings

The refinement moment.



This isn't a derby problem



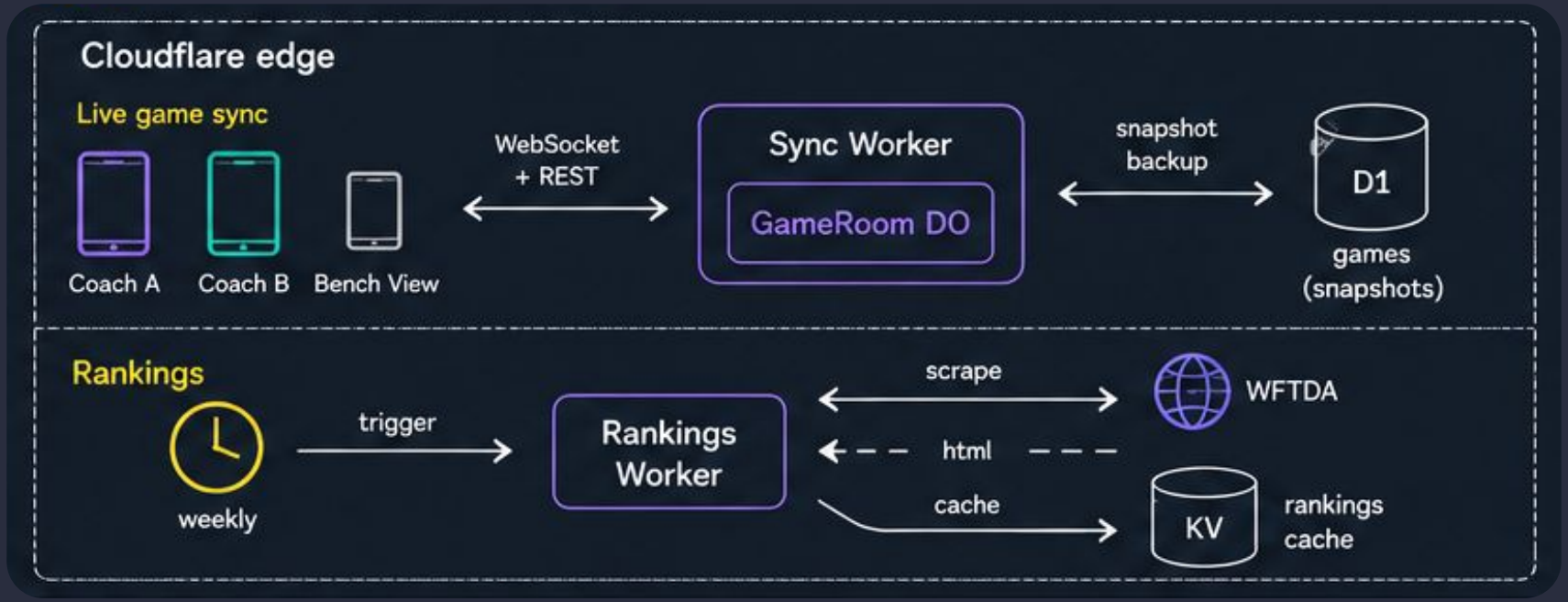
Same architecture. Same patterns.

Two tablets.
One bench.
Zero login.

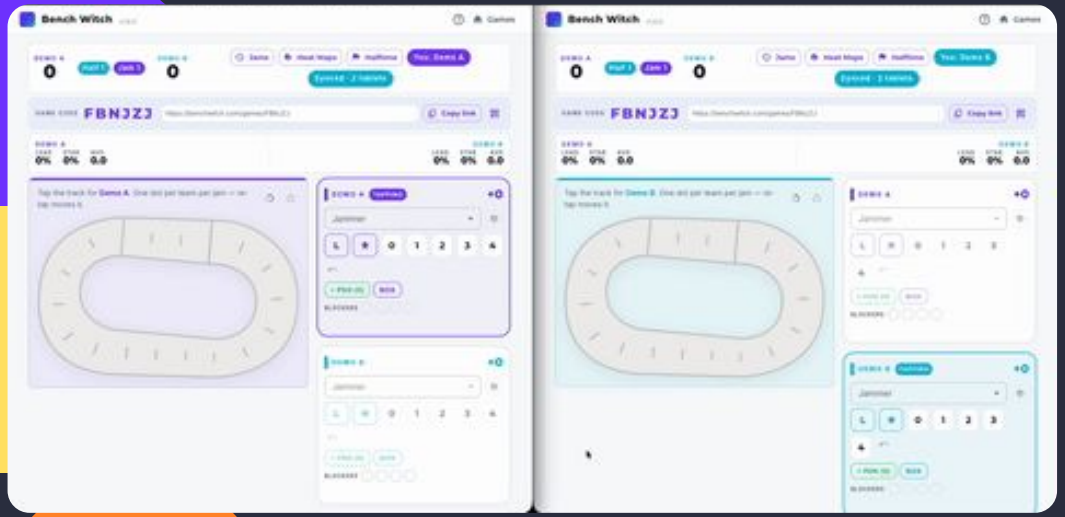
DESIGN CONSTRAINT

Everything I built had to earn this.

Architecture at 30,000 feet.



The GameRoom Durable Object.



One DO Per Game

Named by Game ID.

Holds the canonical snapshot

In memory, ready to go.

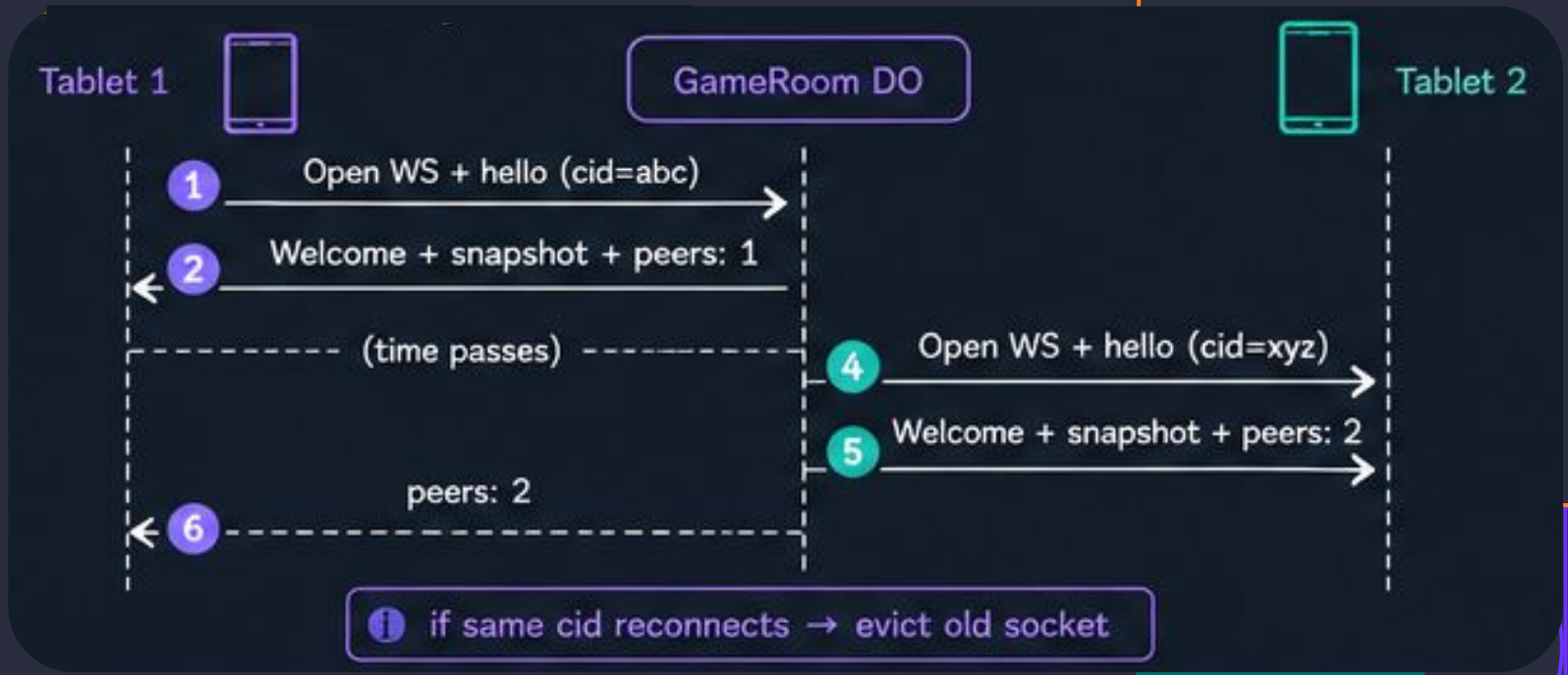
Cold-start rehydrates

From DO Storage, then D1.

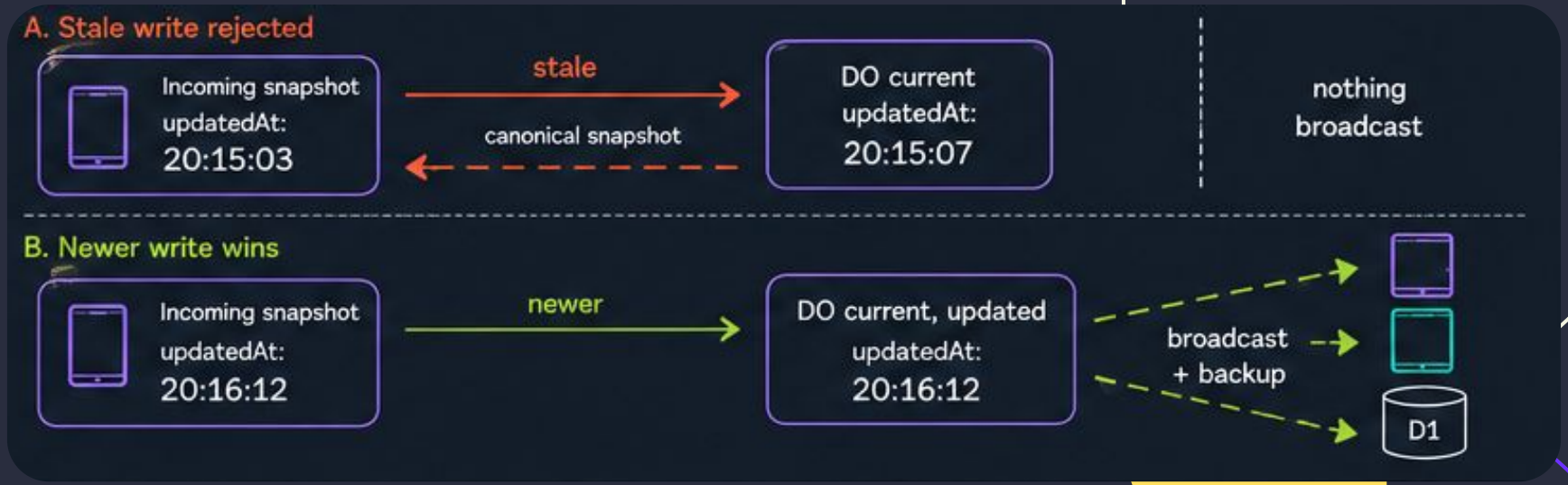
Broadcasts to all peers

WebSocket fan-out.

WebSocket Join Flow



Timestamps Decide Who Wins



The One Method That Carries it All

```
private async setSnapshot(game: unknown): Promise<boolean> {
  const incomingTs = (game as any)?.updatedAt
  const currentTs  = (this.snapshot as any)?.updatedAt

  // 1. Reject stale writes
  if (this.snapshot && incomingTs && currentTs && incomingTs <
currentTs) {
    return false
  }

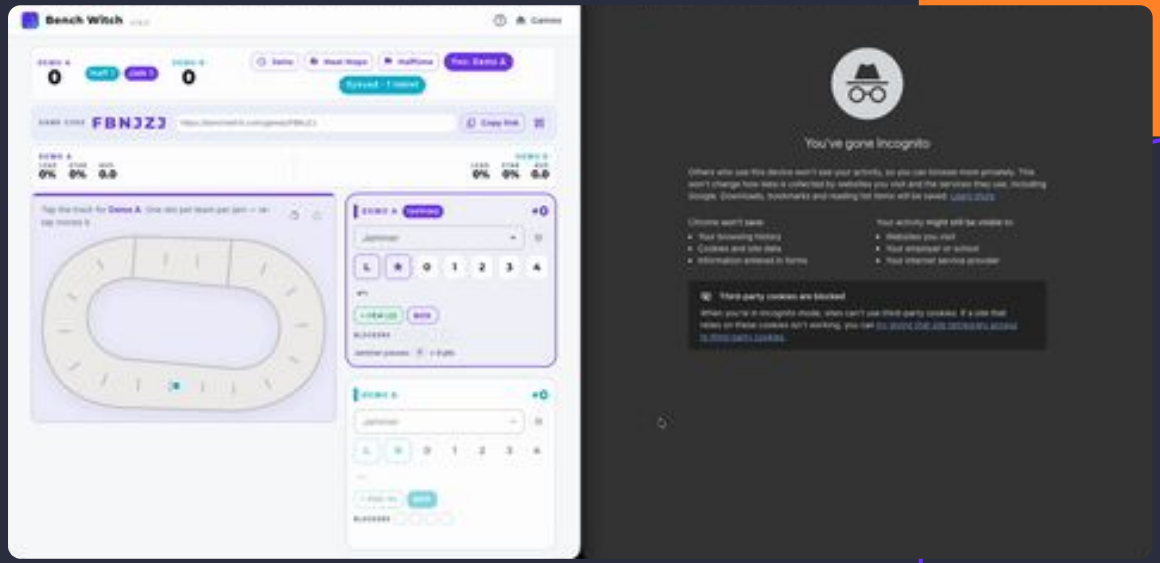
  // 2. Protect server-owned fields (AI carried forward)
  const incoming = game as any
  const existing = this.snapshot as any
  if (incoming && existing?.analysis) {
    incoming.analysis        = existing.analysis
    incoming.analysisVersion = existing.analysisVersion
  }

  // 3. Fast primary write to DO storage
  this.snapshot = incoming
  await this.state.storage.put('snapshot', incoming)

  // 4. Best-effort D1 backup — fire and forget
  if (this.env.DB) {
    this.state.waitUntil(this.env.DB.prepare( ... ).run())
  }

  return true
}
```

Share the game. No Login.



QR + 6-character code

Second version — stripped confusable characters (0/O) after first bench use.

Every game syncs to a library

D1 backs the /api/games endpoint. Home screen shows live first.

No auth. Ever.

Built for one team. If you have the code, you have the game.

Rankings Worker + Recap



Six Design Constraints

Track First

Big Targets

One Tap › Two

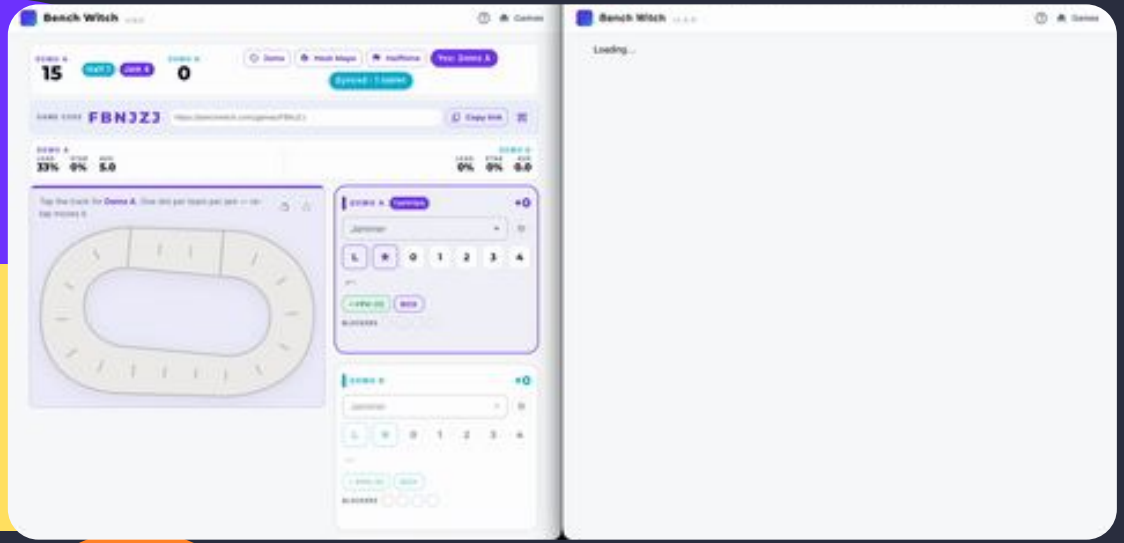
Glanceable

Forgiving

Light & Offline

These two do most of the work in this section.

Late Joins.



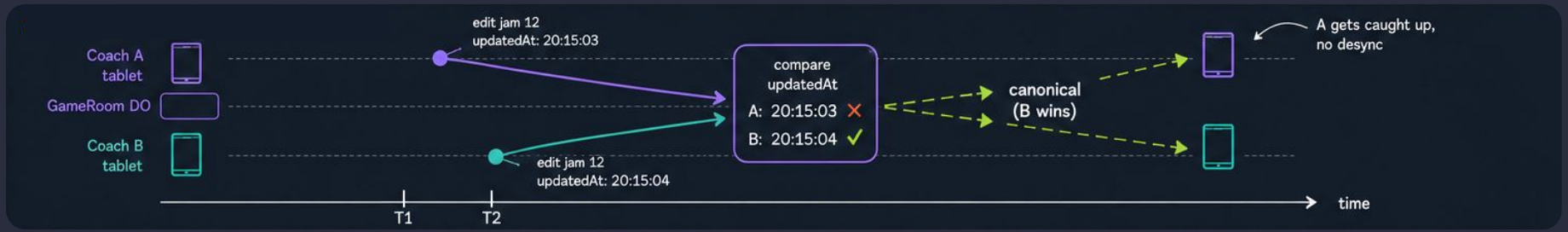
New Tablet Joins
Opens the share link mid-jam.

Role Picker Appears
Which team? Pick one.

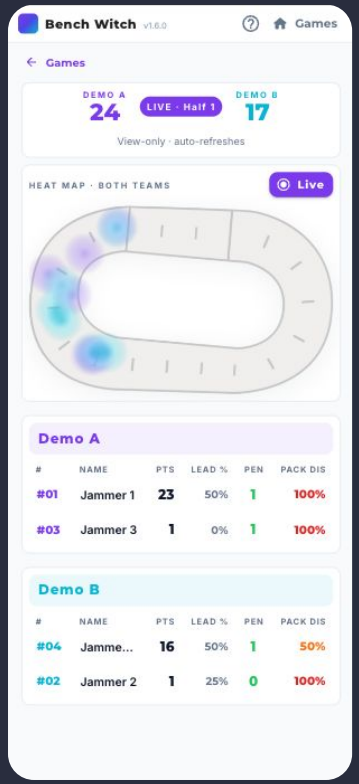
Snapshot Pushed Instantly
Every jam so far, current score.

No “Catching Up” Moment
The DO always has it ready.

Conflicting Writes.



The Read-only Third Device

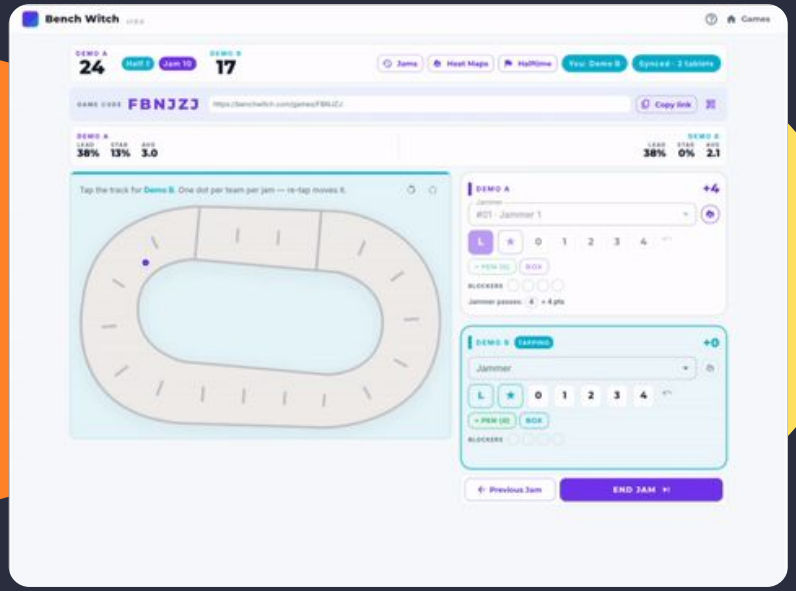


Same Game Code
Everyone can open it.

No Write Path
Zero chance of accidental clobber.

Auto-refreshes
As tag-team coaches edit.

For Whoever wanders over
The third tablet from the title.



Disconnect and Reconnect

In-flight in the DO

Live memory holds the snapshot.

At Rest in DO Storage

Cloudflare persists it.

Backed up in D1

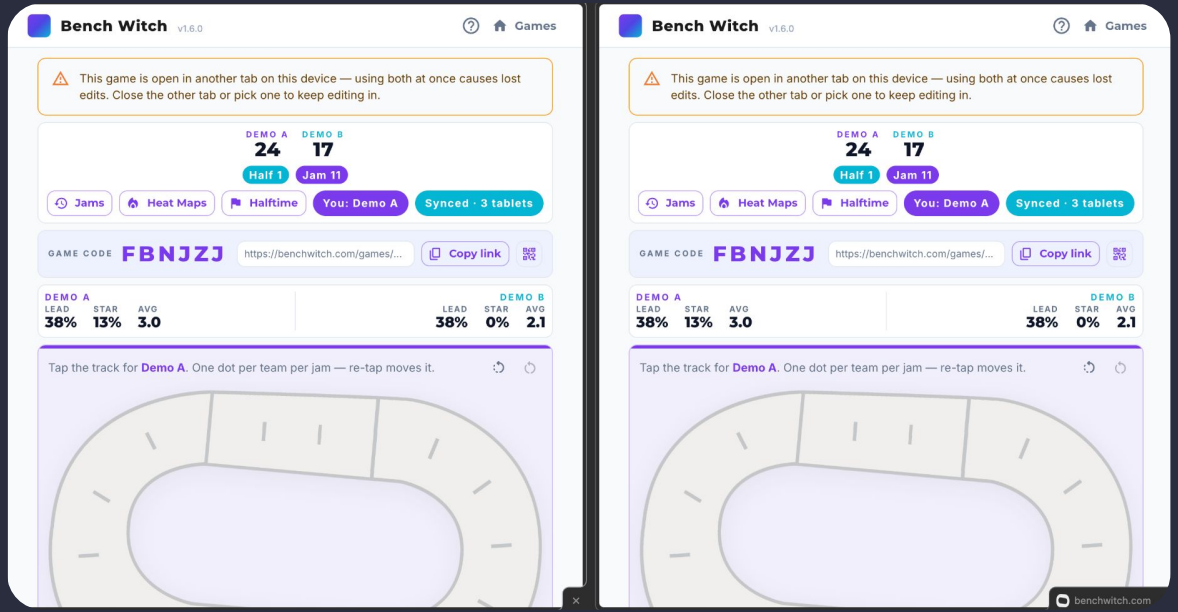
Even if the DO evicts.

Same Client ID Reconnects

No 'three tablets' embarrassment.

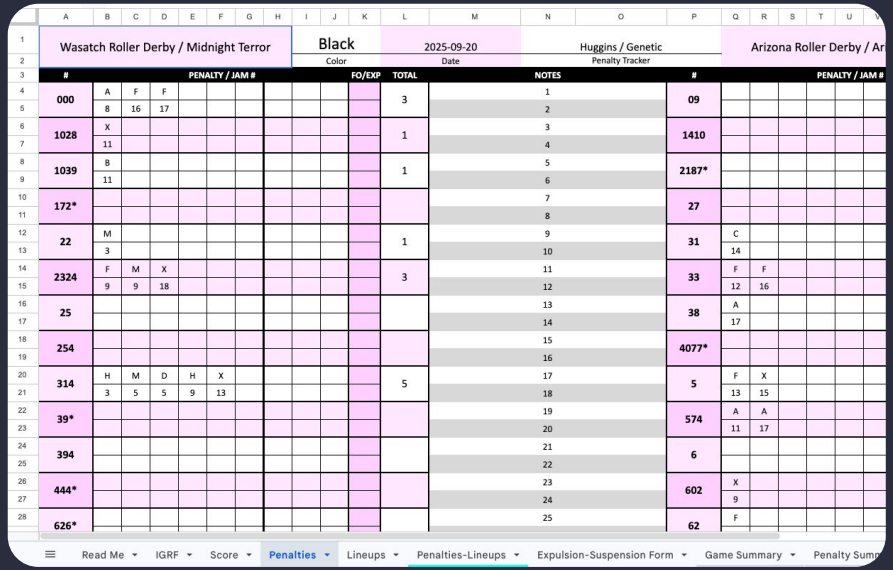
The Two-Tab Guard.

Doesn’t try to be clever. Just tells you.



The problem I couldn’t solve with more data.

Coaches wanted a paragraph, not another table.



| | | |

Ten Lines to Call the Model

```
const result = await this.env.AI.run(
'@cf/meta/llama-3.3-70b-instruct-fp8-fast',
  {
    messages: [
      {
          role: 'system',
          content: ANALYSIS_SYSTEM_PROMPT
     },
     { role: 'user', content: prompt },
    ],
    max_tokens: 800,
  },
) as { response?: string }
```

Fires only when a bout is finalized

Never during live play.

Version-tagged prompt

Bump version → regenerate on next open.

Server owns the field

Clients never write to it.

Why llama, not Claude/GPT?

Native Cloudflare integration — no separate key, account, or bill.

The prompt is where the work is

```
const ANALYSIS_SYSTEM_PROMPT = [
  'Tone: analytical and observational, like a stats-driven
beat writer. ' +
    'Connect facts into patterns; let the reader draw their
own conclusions.',

  'Allowed comparative phrasing: ' +
    '"the most penalties on A (5)", ' +
    '"the lowest lead percentage among A\'s 3+ jam jammers", '
+
    '"took 4 of their 6 leads after halftime"…',

  'Do NOT use these praise words: ' +
    '"stood out", "impressive", "strong", "notable", "shined",
' +
    '"dominant", "key", "effective", "best", "carried",
"anchored".',

  'Do NOT use these criticism words: ' +
    '"struggled", "weak", "poor", "concerning", "issue",
"trouble", ' +
    '"ineffective", "difficulty", "failed",
"underperformed".',

  'When making a comparative claim ("most X", "highest Y",
"lowest Z"), ' +
    'you MUST use only the values from the STANDOUTS block. '
+
    'Never compute superlatives yourself; only cite them.',
].join(' ')
```

```
// Excerpt of the per-bout prompt handed to the model
const prompt = `
STANDOUTS — pre-computed superlatives.
Cite these exactly; do not invent your own "most/least"
claims:

Team A (All-Stars):
  - Most points:    #14 Jade (32)
  - Most leads:     #12 Razor (7 of 12, 58%)
  - Most penalties: #22 Killjoy (5)
  - Highest pack-disadvantage: #22 Killjoy (5 of 8 jams, 62%)

Team B (Riveters):
  - Most points:    #393 Kato Baelin (24)
  - Most leads:     #179 Sue-Nami (5 of 10, 50%)
  - Most penalties: #408 Purr Oxide (3)
`
```

What can go wrong, and what I do.

Empty Response

Catch it. Don't write the field. Log. Try again on next open.

Rate Limits/Model Errors

Same story. Caught, logged, next attempt covers it.

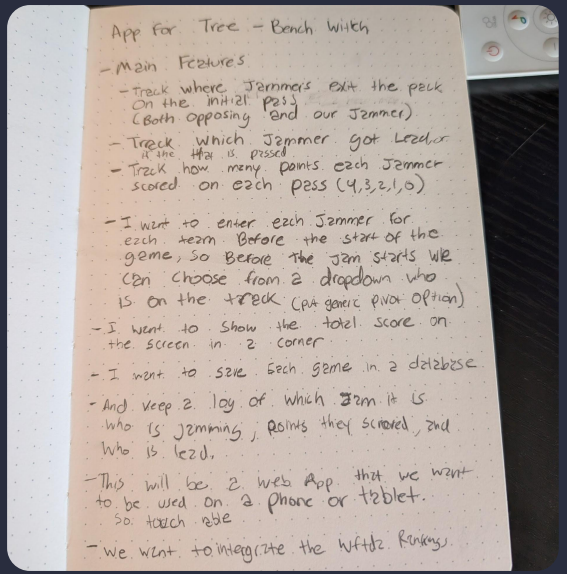
Concurrent runs

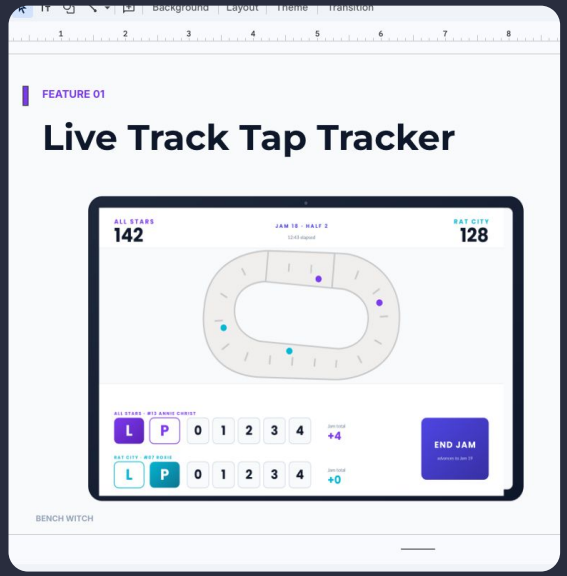
In-flight guard. First call wins. Second one skips out.

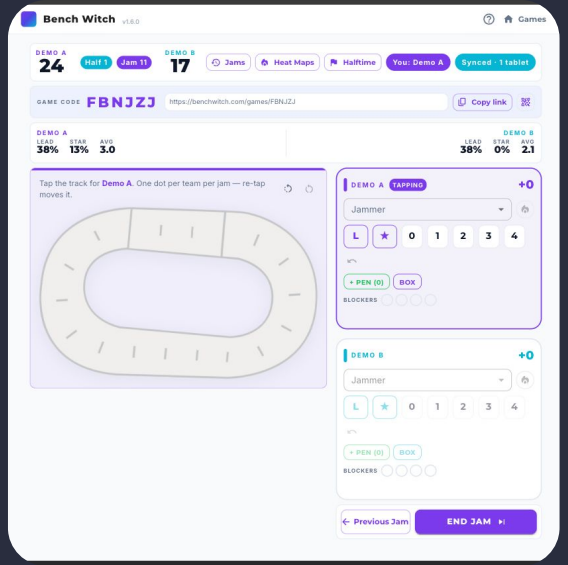
Cost Creep

Under $1/mo at my scale. Won't hold if this scales — holds for me.

Notebook to Production. Thirty Days.







Playoffs in 30 days set the pace. AI is a force multiplier on clarity.

What AI actually accelerated.

The Design Deck

18 slides in one night.

Brand, tech stack, data model, six UX principles, build plan. A week of solo work became an evening.

Cloudflare Boilerplate

The stuff nobody wants to type.

WebSocket handlers, CORS headers, Durable Object plumbing, MUI theme setup. Every project needs it. AI just made it disappear.

Prompt Iteration

12 versions of the analysis prompt.

AI helping me iterate on AI prompts is a weird loop. Each rewrite got tighter, cleaner, closer to what coaches actually wanted.

CI/CD YAML

The indentation I always forget.

GitHub Actions for build, test, deploy to Cloudflare. Because `secrets.CLOUDFLARE_API_TOKEN` never goes where I think it should.

Context Up Front

Fed Claude the rule book.

WFTDA rules and WCAG accessibility guidelines, loaded once. Every prompt after that went in with the domain and the standards already there.



What AI Couldn’t Help With

Judgment problems, not generation problems.

Domain Modeling

AI thinks pivots are gymnastics and jams are music.

The 'factual fact-only' tone

12 versions of the prompt to earn.

Which edge cases actually matter

You have to ship and watch coaches use it.

Design language decisions

No all-caps buttons. Foul-out threshold at 7. Big gloved-hand targets.

The story that lands, not the story that hypes.

One night to v1 → because playoffs started in 30 days.
Every version since → shipped from what real games revealed.

AI didn't build the app. It collapsed the friction between deciding and building.



Four patterns you can steal.

Design Constraints First

Architecture second.
Always.

DO + D1 + WebSockets

Authoritative in-flight store,
best-effort durable backup.

AI as a Summary Layer

Not a decision layer.
Pre-compute the math.
Constrain the vocabulary.

Ship for one Real User

Iterate on what real use
reveals. AI compresses the
pipeline; iteration wins.

Thanks.

Want to talk about any of this? DM me.

Thanks to my coaching staff — who let me build them a tool, and patiently helped me iterate on it every game.

And thanks to the derby community, who made a niche sport worth building for.

@akendalb

LinkedIn



Thank you

Amanda Kendal-Brown
DevFest DC 2026